

BÁO CÁO DỰ ÁN

Hệ thống SecureChat

NGUYỄN VŨ THÀNH

Thanh14704@gmail.com

Lĩnh Vực: Kỹ Sư Phần Mềm: Lập Trình Hệ Thống

Mentor hướng dẫn: Lê Thành Thân

Đơn vị: Viettel Cypher Security

Chương trình: Viettel Digital Talent

HÀ NỘI, 6/2026

TÓM TẮT NỘI DUNG DỰ ÁN

Dự án trình bày việc thiết kế và phát triển hệ thống SecureChat - một giải pháp truyền thông thời gian thực (Real-time Communication) theo mô hình Client-Server. Hệ thống được xây dựng hoàn toàn bằng ngôn ngữ C++ với mục tiêu đáp ứng đồng thời hai yêu cầu : Hiệu năng xử lý đồng thời cao ở mức cao (High Concurrency) và Bảo Mật thông tin đa lớp.

Ở phía Máy chủ (Server), kiến trúc hệ thống được thiết kế dựa trên mô hình điều khiển sự kiện bất đồng bộ (Event-Driven) sử dụng *epoll*, kết hợp cùng mô hình Thread Pool. Điều này cho phép Server xử lý bài toán C10K một cách tối ưu, không bị thất nút cổ chai tại các luồng I/O mạng. Cơ sở dữ liệu SQLite được tích hợp cùng cơ chế Write-Ahead Logging (WAL) giúp đảm bảo tốc độ truy vấn thông tin người dùng và lịch sử hệ thống trong môi trường đa luồng.

Về mặt Giao thức và Bảo mật, hệ thống không sử dụng các giao thức văn bản truyền thống mà lựa chọn sử dụng giao thức truyền tải nhị phân tùy chỉnh (Custom Binary Protocol) với Header chứa mã kiểm lỗi CRC32. Quá trình bảo mật áp dụng mô hình mã hóa lai (Hybrid Crypto): sử dụng RSA-2048 để trao đổi khóa phiên một cách an toàn ở bước bắt tay (Handshake), sau đó toàn bộ payload giao tiếp được mã hóa và xác thực tính nguyên vẹn bằng chuẩn AES-256-GCM.

Ngoài ra, dự án tập trung nghiên cứu và cài đặt một dịch vụ an ninh mạng chủ động ngay trong nhân máy chủ, bao gồm: module Hệ thống phát hiện xâm nhập (Intrusion Detector - IDS) để phân tích hành vi độc hại, bộ định tuyến giới hạn tốc độ (Rate Limiter) chống lại các cuộc tấn công DoS/Spam dựa trên thuật toán Token-Bucket, và Nhật ký kiểm tra (Audit Logger) lưu vết các thao tác nhạy cảm. Phía Client cài đặt giao diện dòng lệnh (CLI) nhẹ, hỗ trợ đa dạng tính năng nghiệp vụ từ nhắn tin cá nhân, phòng chat nhóm, truyền tải tập tin, đến các tập lệnh của quản trị viên (Admin Commands).

Kết quả đạt được là một nền tảng giao tiếp tin cậy, hoạt động ổn định và đáp ứng các tiêu chuẩn bảo mật thực tế, có khả năng mở rộng và tích hợp thêm các dịch vụ khác trong tương lai.

MỤC LỤC

Danh sách hình vẽ	iii
Danh sách bảng	iv
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Khảo sát hiện trạng	1
1.1.1 Đặt vấn đề	1
1.1.2 Mục tiêu và phạm vi của đề tài	1
1.2 Phân tích yêu cầu hệ thống	2
1.2.1 Yêu cầu chức năng	2
1.2.2 Yêu cầu phi chức năng	3
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	4
2.1 Kiến trúc tổng thể và Phân tích yêu cầu	4
2.1.1 Phân tích yêu cầu hệ thống	4
2.1.2 Kiến trúc tổng thể	5
2.2 Giao thức Mạng và Mô hình I/O	7
2.2.1 Mô hình Client-Server và I/O Multiplexing	7
2.2.2 Thiết kế Định dạng Gói tin	7
2.3 Thiết kế Cơ sở dữ liệu và Cơ chế Xác thực	9
2.3.1 Cấu trúc Lưu trữ Dữ liệu	9
2.3.2 Luồng Xác thực và Quản lý Phiên	10
2.4 Giao thức Mật mã và Tính bảo mật	11
2.4.1 Luồng Trao đổi khóa	11
2.5 Hệ thống Chức năng Cốt lõi	11
2.5.1 Quản lý Phòng và Định tuyến Thông điệp	11
2.5.2 Trung chuyển Tập tin Ẩn danh	13
2.6 Hệ thống Quản trị và Chống lạm dụng	15
2.6.1 Phòng chống Tấn công Từ chối Dịch vụ (DoS/DDoS)	15
2.6.2 Kiểm soát Truy cập và Quản trị (RBAC)	17
CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	19

3.1	Môi trường và Công nghệ Triển khai	19
3.1.1	Cấu hình Môi trường Hệ thống	19
3.1.2	Kịch bản Biên dịch và Vận hành	20
3.2	Kịch bản Kiểm thử và Đánh giá Hiệu năng	23
3.2.1	Tổng quan Hệ thống Kiểm thử Tự động	23
3.2.2	Đánh giá Giới hạn Chịu tải (Load Benchmark)	25
3.3	Đánh giá Khả năng Phòng thủ Hệ thống	26
3.3.1	Phòng thủ trước tấn công Từ chối dịch vụ	26
3.3.2	Phòng thủ trước tấn công Tiêm nhiễm và Vượt quyền	28
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		30
4.1	Kết quả đạt được	30
4.2	Hướng phát triển tương lai	31

Danh sách hình vẽ

Hình 2.1:	Sơ đồ Component Kiến trúc Tổng thể Hệ thống	5
Hình 2.2:	Sơ đồ Thực thể Liên kết của hệ thống CSDL	9
Hình 2.3:	Lược đồ Trình tự luồng Xác thực và Khôi phục Phiên	10
Hình 2.4:	Lược đồ trình tự Bắt tay trao đổi khóa lai RSA-AES	11
Hình 2.5:	Sơ đồ Hoạt động Định tuyến Thông điệp và Quản lý Phòng	13
Hình 2.6:	Máy trạng thái hữu hạn luồng Truyền tệp ẩn danh	15
Hình 2.7:	Lược đồ Khối Hệ thống Tường lửa mềm	16

Danh sách bảng

Bảng 2.1:	Đặc tả yêu cầu chức năng hệ thống	4
Bảng 2.2:	Đặc tả yêu cầu phi chức năng hệ thống	5
Bảng 2.3:	Phân rã các thành phần kiến trúc hệ thống	6
Bảng 2.4:	Cấu trúc Gói tin Nhị phân	8
Bảng 2.5:	Đặc tả các Bảng thực thể trong Cơ sở dữ liệu	9
Bảng 2.6:	Chiến lược cấp phát khóa trong RoomManager	13
Bảng 2.7:	Các trạng thái của Phiên truyền tệp	14
Bảng 2.8:	Hệ thống Điểm phạt Hành vi	16
Bảng 2.9:	Ma trận Phân quyền của Hệ thống	18
Bảng 3.1:	Bảng Ma trận Phụ thuộc Thư viện	20
Bảng 3.2:	Các tập lệnh và tham số cấu hình trong quy trình biên dịch	21
Bảng 3.3:	Các tham số dòng lệnh vận hành Máy chủ	22
Bảng 3.4:	Các tham số dòng lệnh Máy khách	23
Bảng 3.5:	Ma trận phân bổ các module kiểm thử tự động	24
Bảng 3.6:	Đo lường Thông lượng và Độ trễ của Server	26
Bảng 3.7:	Cấu hình giới hạn tốc độ của hệ thống	28
Bảng 3.8:	Ma trận điểm số rủi ro của hệ thống phòng thủ	29

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
ACK	Acknowledgment
AD	Authenticated Data
AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
CLI	Command-Line Interface
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DoS	Denial-of-Service
DDoS	Distributed Denial-of-Service
ERD	Entity-Relationship Diagram
ET	Edge-Triggered
FD	File Descriptor
GCM	Galois/Counter Mode
GUI	Graphical User Interface
HIDS	Host-based Intrusion Detection System
HMAC	Hash-based Message Authentication Code
I/O	Input/Output
IDS	Intrusion Detection System
IP	Internet Protocol
LT	Level-Triggered
MTU	Maximum Transmission Unit
OOM	Out of Memory
RSA	Rivest-Shamir-Adleman
SHA	Secure Hash Algorithm
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
WAL	Write-Ahead Logging

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Khảo sát hiện trạng

1.1.1 Đặt vấn đề

Sự phát triển của hạ tầng mạng thúc đẩy nhu cầu giao tiếp trực tuyến. Máy chủ (Chat Server) là thành phần cốt lõi của các ứng dụng truyền thông thời gian thực (Real-time Communication), thực hiện chức năng định tuyến thông điệp song công (Full-duplex) giữa các máy khách (Client).

Quá trình xử lý tải cao đặt ra yêu cầu về tối ưu hiệu năng. Mô hình cấp phát luồng theo kết nối (Thread-per-client) bộc lộ hạn chế đối với bài toán đa kết nối đồng thời (C10K), gây thắt cổ chai I/O (I/O Bottleneck) và lãng phí tài nguyên. Ngoài ra, giao thức văn bản (Text-based Protocol) phát sinh chi phí phân tích cú pháp (Parsing overhead) và tiêu tốn băng thông. Giải pháp kỹ thuật được đề xuất là kết hợp mô hình I/O đa kênh (I/O Multiplexing) và giao thức nhị phân (Binary Protocol) nhằm tối ưu băng thông và giảm độ trễ.

Tính an toàn của dữ liệu truyền tải là yêu cầu bắt buộc. Sự thiếu hụt các cơ chế kiểm soát truy cập và bảo vệ kênh truyền dẫn đến rủi ro từ các cuộc tấn công nghe lén (Eavesdropping), tấn công xen giữa (MitM) và từ chối dịch vụ (DoS/Spam). Do đó, hệ thống cần tích hợp các module bảo mật như mã hóa dữ liệu, hệ thống phát hiện xâm nhập (IDS) và giới hạn lưu lượng (Rate Limiter).

Dựa trên các yêu cầu này, đề tài tiến hành nghiên cứu và phát triển hệ thống máy chủ trò chuyện bảo mật (Secure Chat Server) hiệu năng cao, tập trung vào tối ưu hóa I/O mạng, định tuyến thông điệp và triển khai các cơ chế bảo mật.

1.1.2 Mục tiêu và phạm vi của đề tài

1. Mục tiêu của đề tài

Đề tài nghiên cứu và phát triển hệ thống máy chủ truyền thông (Secure Chat Server) theo mô hình Client-Server với các mục tiêu:

- Tối ưu hiệu năng: Sử dụng I/O Multiplexing (epoll) và Thread Pool nhằm giải quyết bài toán C10K, tối ưu hóa quá trình xử lý I/O và giảm độ trễ.
- Tối ưu giao thức truyền tải: Áp dụng giao thức nhị phân tùy chỉnh (Custom Binary Protocol) để loại bỏ chi phí phân tích cú pháp và tiết kiệm băng thông.
- Bảo mật hệ thống: Triển khai mã hóa lai (RSA-2048, AES-256-GCM) và tích hợp các module bảo mật (IDS, Rate Limiter bằng thuật toán Token-Bucket, Audit Logger).

- Đảm bảo tính nhất quán dữ liệu: Sử dụng SQLite kết hợp cơ chế Write-Ahead Logging (WAL) để tối ưu truy vấn đa luồng.
2. Phạm vi nghiên cứu và triển khai
- Phạm vi nghiên cứu và giới hạn của đề tài bao gồm:
- Kiến trúc hệ thống: Lõi máy chủ (Server-side) được xây dựng bằng C++. Phần mềm máy khách (Client-side) sử dụng giao diện dòng lệnh (CLI) để kiểm thử (không phát triển GUI hay ứng dụng di động).
 - Chức năng nghiệp vụ: Triển khai nhắn tin cá nhân (Direct Message), phòng nhóm (Rooms), truyền tệp (File Transfer) và các lệnh quản trị (Admin Commands).
 - Kiểm định và đánh giá: Đo lường hiệu năng (Benchmark) dưới tải cao và giả lập các kịch bản tấn công (DoS, quét cổng, giả mạo gói tin) để kiểm chứng khả năng phòng thủ.

1.2 Phân tích yêu cầu hệ thống

1.2.1 Yêu cầu chức năng

Hệ thống yêu cầu đáp ứng các nhóm chức năng cốt lõi sau:

1. Quản lý tài khoản và xác thực (Authentication & Authorization):
 - Đăng ký (Register) và đăng nhập (Login) an toàn. Mật khẩu được băm (SHA256/bcrypt) trước khi lưu trữ.
 - Cấp phát thẻ định danh phiên (Session Token) bằng HMAC, hỗ trợ tự động phục hồi kết nối (Session Resumption).
2. Quản lý phiên kết nối và trạng thái (Session Management):
 - Quản lý vòng đời kết nối mạng (Connect, Disconnect) và duy trì tín hiệu sống sót (Ping/Pong).
 - Tra cứu danh sách người dùng trực tuyến (List Users) và thông tin hồ sơ (Whois).
3. Giao tiếp và nhắn tin thời gian thực (Real-time Chatting):
 - Nhắn tin cá nhân (Direct Message): Trao đổi thông điệp trực tiếp (1-1).
 - Nhắn tin nhóm (Room Chat): Liệt kê phòng (List Rooms), tạo phòng (Create Room), tham gia (Join), rời phòng (Leave) và phát đa hướng thông điệp (Broadcast).
 - Truy vấn lịch sử trò chuyện từ cơ sở dữ liệu.
4. Truyền tải tệp tin (File Transfer):
 - Hỗ trợ luồng điều khiển truyền tệp hai chiều: gửi yêu cầu (File Request) và xác nhận (Accept/Reject).
 - Truyền dữ liệu dạng phân mảnh (Chunking) để chống tắc nghẽn luồng thông điệp chính.
5. Quản trị hệ thống (Admin Commands):
 - Áp dụng cơ chế phân quyền quản trị viên (Admin Privilege).

- Xử lý vi phạm: kích xuất (Kick), cấm gửi tin (Mute), cấm truy cập (Ban IP/Username), thăng cấp (Promote) và phát thông báo hệ thống (System Broadcast).
6. An ninh và phòng vệ chủ động (Security):
- Bắt tay mật mã (Crypto Handshake): Trao đổi khóa (Key Exchange) khi khởi tạo kết nối để thiết lập kênh truyền bảo mật.
 - Giới hạn lưu lượng (Rate Limiting): Phòng chống tấn công từ chối dịch vụ (DoS) và thư rác (Spam).
 - Hệ thống phát hiện xâm nhập (IDS): Phân loại và ghi nhận vi phạm (sai mật khẩu, quét cổng, gói tin dị dạng) để áp dụng biện pháp khóa IP (Ban).

1.2.2 Yêu cầu phi chức năng

Các yêu cầu phi chức năng nhằm đảm bảo khả năng vận hành, độ tin cậy và an toàn thông tin:

1. Hiệu năng và khả năng mở rộng (Performance & Scalability):
 - Xử lý đồng thời: Sử dụng kiến trúc I/O đa kênh (I/O Multiplexing) với epoll và Thread Pool nhằm duy trì luồng điều phối phi đồng bộ (Non-blocking). Giải quyết bài toán C10K với độ trễ thấp.
 - Tối ưu băng thông: Áp dụng giao thức nhị phân (Custom Binary Protocol) để tối thiểu hóa kích thước gói tin và hao phí phân tích cú pháp (Parsing).
2. Bảo mật và toàn vẹn dữ liệu (Security & Integrity):
 - Mã hóa kênh truyền: Mã hóa tải trọng bằng AES-256-GCM, trao đổi khóa phiên (Session Key) qua RSA-2048.
 - Toàn vẹn thông điệp: Áp dụng kiểm lỗi CRC32 và mã xác thực HMAC cho từng gói tin.
 - An toàn lưu trữ: Băm mật khẩu một chiều trước khi lưu trữ. Sử dụng Write-Ahead Logging (WAL) cho cơ sở dữ liệu SQLite để duy trì tính nhất quán.
3. Tính tin cậy và khả năng chịu lỗi (Reliability & Fault Tolerance):
 - Quản lý tài nguyên: Tự động thu hồi tài nguyên (socket, phiên đăng nhập, vùng nhớ) của kết nối vi phạm thời gian chờ (Timeout) hoặc ngắt đột ngột, ngăn chặn cạn kiệt mô tả tệp (File Descriptor Exhaustion) và rò rỉ bộ nhớ (Memory Leak).
 - Xử lý luồng mạng: Triển khai bộ đệm (Network Buffer) độc lập cho từng kết nối nhằm xử lý hiện tượng phân mảnh (TCP Fragmentation) và dính gói (TCP Sticky Packets).

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày chi tiết kiến trúc tổng thể và các quyết định thiết kế cốt lõi của hệ thống. Quá trình phân tích và thiết kế được gắn kết chặt chẽ với các nền tảng lý thuyết về mạng máy tính, mô hình vào/ra (I/O) và mật mã học, nhằm làm rõ cơ chế vận hành của từng phân hệ (module) trong kiến trúc thực tế.

2.1 Kiến trúc tổng thể và Phân tích yêu cầu

2.1.1 Phân tích yêu cầu hệ thống

Quá trình phân tích xác định các tiêu chí thiết kế cho máy chủ truyền thông. Yêu cầu hệ thống bao gồm hai nhóm chính: chức năng và phi chức năng.

1. Yêu cầu chức năng

Bảng 2.1 đặc tả các chức năng đối với Người dùng (User) và Quản trị viên (Admin).

Bảng 2.1: Đặc tả yêu cầu chức năng hệ thống

Nhóm chức năng	Tác nhân	Mô tả chi tiết
Xác thực định danh	Người dùng	Đăng ký, đăng nhập và quản lý vòng đời phiên làm việc.
Nhắn tin trực tiếp	Người dùng	Truyền thông điệp song công (Full-duplex) 1-1.
Quản lý phòng trò chuyện	Người dùng	Khởi tạo, tham gia, quản lý phòng và kiểm soát truy cập.
Truyền tải tệp tin	Người dùng	Truyền nhận tệp đa phương tiện bằng cơ chế phân mảnh (Data Chunking).
Quản trị hệ thống	Quản trị viên	Kiểm soát truy cập (RBAC), quản lý tài khoản và giám sát.
Xử lý vi phạm	Quản trị viên	Cấm gửi tin, ngắt kết nối và khóa tài khoản.

2. Yêu cầu phi chức năng

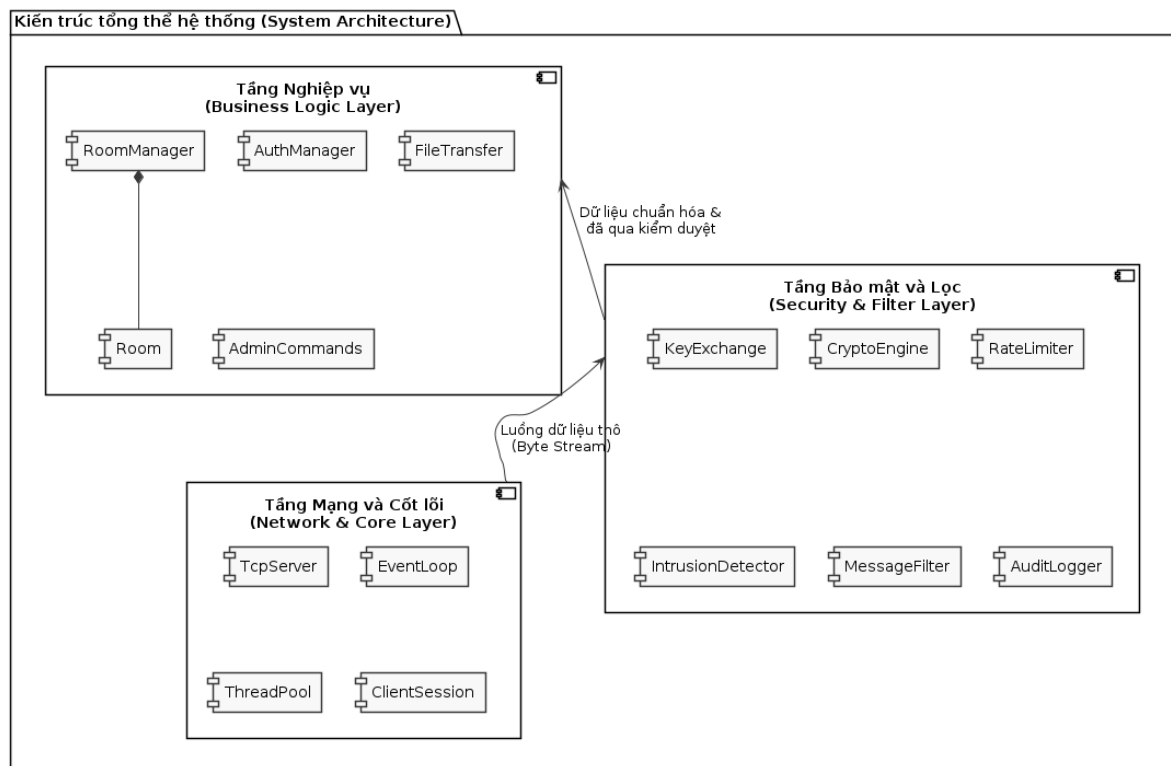
Bảng 2.2 trình bày các ràng buộc kỹ thuật về hiệu năng, an toàn và bảo mật hệ thống.

Bảng 2.2: Đặc tả yêu cầu phi chức năng hệ thống

Tiêu chí	Mô tả chi tiết
Hiệu năng và Xử lý đồng thời	Đảm bảo tính sẵn sàng, độ trễ thấp. Xử lý kết nối quy mô lớn (C10K) bằng I/O phi đồng bộ và Thread Pool/Event Loop.
Bảo mật và Mã hóa	Mã hóa đầu cuối (E2EE) cho luồng dữ liệu truyền tải qua TLS/SSL.
An toàn mạng	Giới hạn tỷ lệ và giám sát lưu lượng nhằm chống DoS/DDoS, Brute-force và Spam.
Nhật ký và Kiểm toán	Lưu trữ nhật ký hệ thống và bảo mật phục vụ truy vết.

2.1.2 Kiến trúc tổng thể (System Architecture)

Kiến trúc máy chủ sử dụng mô hình phân tầng (Layered Architecture) và nguyên lý tách biệt trách nhiệm (Separation of Concerns). Hệ thống bao gồm 3 phân tầng độc lập, thực hiện xử lý từ mức dữ liệu thô đến logic nghiệp vụ.



Hình 2.1: Sơ đồ Component Kiến trúc Tổng thể Hệ thống

Bảng 2.3 mô tả chi tiết thành phần của các phân tầng.

Bảng 2.3: Phân rã các thành phần kiến trúc hệ thống

Phân tầng	Thành phần	Chức năng cốt lõi
Tầng Mạng và Cốt lõi	TcpServer	Quản lý kết nối TCP.
	EventLoop	Điều phối sự kiện mạng bất đồng bộ.
	ThreadPool	Xử lý tác vụ tính toán (CPU-bound).
	ClientSession	Duy trì trạng thái phiên kết nối.
Tầng Bảo mật và Lọc	KeyExchange	Trao đổi khóa phiên.
	CryptoEngine	Mã hóa và giải mã luồng dữ liệu.
	RateLimiter	Kiểm soát tốc độ truy cập.
	IntrusionDetector	Phát hiện tấn công.
	MessageFilter	Kiểm duyệt dữ liệu đầu vào.
	AuditLogger	Lưu vết sự kiện bảo mật.
Tầng Nghiệp vụ	AuthManager	Xác thực và quản lý phiên làm việc.
	RoomManager	Quản lý vòng đời phòng trò chuyện.
	Room	Quản lý trạng thái phòng và đa hướng thông điệp.
	FileTransfer	Điều phối phân mảnh tệp dữ liệu.
	AdminCommands	Thực thi tập lệnh quản trị.

Cơ chế tương tác giữa các phân tầng:

1. Tầng Mạng và Cốt lõi

Tương tác với Socket API theo mô hình phi đồng bộ để xử lý I/O mạng.

- TcpServer: Lắng nghe kết nối TCP và cấp phát tài nguyên.
- EventLoop: Điều phối sự kiện mạng thông qua I/O đa kênh.
- ThreadPool: Xử lý các tác vụ CPU-bound.
- ClientSession: Quản lý trạng thái phiên, bộ đệm và tham chiếu khóa.

2. Tầng Bảo mật và Lọc

Kiểm duyệt dữ liệu mạng trước khi chuyển giao cho tầng nghiệp vụ.

- KeyExchange & CryptoEngine: Trao đổi khóa và thực thi mã hóa/giải mã.
- RateLimiter: Kiểm soát tần suất yêu cầu.
- IntrusionDetector & MessageFilter: Phát hiện bất thường và làm sạch dữ liệu đầu vào.
- AuditLogger: Ghi nhận sự kiện hệ thống.

3. Tầng Nghiệp vụ

Thực thi logic ứng dụng.

- AuthManager: Xác thực định danh và quản lý phiên.
- RoomManager & Room: Quản lý vòng đời phòng và định tuyến thông điệp.
- FileTransfer: Phân mảnh và tái tổ hợp dữ liệu tệp.
- AdminCommands: Xử lý lệnh quản trị.

2.2 Giao thức Mạng và Mô hình I/O

2.2.1 Mô hình Client-Server và I/O Multiplexing (epoll)

1. Cơ sở lý thuyết

Mô hình Thread-per-connection phát sinh chi phí chuyển ngữ cảnh (Context Switching) lớn đối với bài toán C10K. Do đó, kỹ thuật I/O Multiplexing với API epoll (Linux) được áp dụng để một luồng có thể giám sát nhiều mô tả tệp (File Descriptor - FD). Thông qua cấu trúc cây đỏ đen (Red-Black Tree) và danh sách liên kết (Linked List), epoll đạt độ phức tạp $\mathcal{O}(1)$ trong quá trình xử lý sự kiện.

2. Triển khai hệ thống

Thành phần TcpServer sử dụng I/O phi đồng bộ (Non-blocking I/O) và epoll để điều phối kết nối TCP theo quy trình sau:

- Khởi tạo điểm cuối: Chuyển trạng thái Socket lắng nghe sang phi đồng bộ thông qua `fcntl`.
- Khởi tạo I/O Multiplexer: Tạo đối tượng epoll (`epoll_create1`) và đăng ký Socket lắng nghe với cờ `EPOLLIN` (`epoll_ctl`).
- Vòng lặp sự kiện: Thực thi `epoll_wait` để thu thập sự kiện I/O.
 - Sự kiện kết nối: Gọi `accept`, chuyển `clientFd` sang trạng thái phi đồng bộ, đăng ký vào epoll (cờ `EPOLLIN` | `EPOLLRDHUP`) và khởi tạo `ClientSession`.
 - Sự kiện dữ liệu: TcpServer tiếp nhận byte thô (`recv`) và giao cho `ClientSession` xử lý.

2.2.2 Thiết kế Định dạng Gói tin (Packet Format)

1. Cơ sở lý thuyết

TCP là giao thức hướng luồng (Stream-oriented), gây ra hiện tượng dính gói (Sticky Packet) và phân mảnh (Half Packet). Tầng ứng dụng sử dụng giao thức nhị phân

(Binary Protocol) để xác định ranh giới gói tin, tối ưu băng thông và giảm chi phí phân tích cú pháp (Parsing Overhead).

2. Cấu trúc gói tin

Hệ thống thiết kế giao thức nhị phân gồm Header (17 Bytes) và Payload, được đặc tả chi tiết tại Bảng 2.4.

Bảng 2.4: Cấu trúc Gói tin Nhị phân

Phân đoạn	Trường dữ liệu	Kích thước	Mô tả chi tiết
Header	<code>magic</code> (Nhận diện)	2 Bytes	Chuỗi byte nhận diện tính hợp lệ.
	<code>version</code> (Phiên bản)	1 Byte	Định danh phiên bản giao thức.
	<code>msg_type</code> (Loại tin)	1 Byte	Định nghĩa nghiệp vụ (đăng nhập, trò chuyện...).
	<code>flags</code> (Cờ điều khiển)	1 Byte	Trạng thái gói tin (mã hóa, nén).
	<code>sequence_num</code> (Thứ tự)	4 Bytes	Hỗ trợ ráp mảnh.
	<code>payload_length</code> (Kích thước)	4 Bytes	Giải quyết hiện tượng Sticky Packet.
	<code>checksum</code> (Mã kiểm tra)	4 Bytes	Xác thực toàn vẹn dữ liệu (Integrity).
Payload	Dữ liệu JSON	Tùy biến	Đóng gói tham số nghiệp vụ (Mã hóa AES-256).

Chu trình phân giải gói tin (Packet Parsing) gồm các bước:

- Đóng gói: Tuần tự hóa JSON, mã hóa và gắn Header (17 Bytes) trước Payload.
- Phân tích Header: `ClientSession` đọc 17 bytes đầu tiên để trích xuất các tham số điều khiển (`msg_type`, `payload_length`, `magic`, `checksum`).
- Đọc Payload: Cấp phát bộ nhớ dựa trên `payload_length` để nhận luồng Payload và xử lý hiện tượng phân mảnh.
- Giải mã và định tuyến: `CryptoEngine` giải mã khối dữ liệu, sau đó hệ thống định

tuyển theo `msg_type` đến bộ xử lý tương ứng.

2.3 Thiết kế Cơ sở dữ liệu và Cơ chế Xác thực

2.3.1 Cấu trúc Lưu trữ Dữ liệu

1. Cơ sở lý thuyết

Hệ quản trị CSDL quan hệ (RDBMS) duy trì tính toàn vẹn thông qua các thuộc tính ACID. Lược đồ dữ liệu (Database Schema) tuân thủ dạng chuẩn 3 (3NF) để tránh dị thường cập nhật (Update Anomalies). Cơ chế Write-Ahead Logging (WAL) được áp dụng để hỗ trợ truy vấn đọc/ghi đồng thời (Concurrency) theo dạng phi chặn (Non-blocking).

2. Triển khai hệ thống

Hệ thống sử dụng SQLite3 (RDBMS nhúng) nhằm giảm độ trễ giao tiếp liên tiến trình (IPC Overhead). Các thiết lập `PRAGMA journal_mode=WAL` và `PRAGMA foreign_keys=ON` lần lượt hỗ trợ truy xuất đồng thời và thực thi toàn vẹn tham chiếu (Referential Integrity). Bảng 2.5 trình bày chi tiết lược đồ dữ liệu với 5 bảng thực thể.

Hình 2.2: Sơ đồ Thực thể Liên kết (ERD) của hệ thống CSDL

Bảng 2.5: Đặc tả các Bảng thực thể trong Cơ sở dữ liệu

Tên Bảng	Khóa & Chỉ mục (Index)	Mô tả chi tiết và Vai trò cốt lõi
Users	PRIMARY KEY (id) UNIQUE (nickname)	Quản lý định danh. Lưu mật khẩu băm (password_hash, salt). Chống Brute-force qua failed_attempts và locked_until.
Rooms	PRIMARY KEY (id) UNIQUE (room_name)	Lưu siêu dữ liệu phòng. Xác thực bằng mật khẩu băm. Khóa ngoại creator_nick tham chiếu Users (ON DELETE CASCADE).
RoomBans	PRIMARY KEY (id) idx_roombans	Danh sách cấm (Blacklist) của phòng. Ràng buộc UNIQUE(room_name, nickname) đảm bảo duy nhất.

Bảng 2.5: Đặc tả các Bảng thực thể trong Cơ sở dữ liệu
(tiếp theo)

Tên Bảng	Khóa & Chỉ mục (Index)	Mô tả chi tiết và Vai trò cốt lõi
ChatHistory	PRIMARY KEY (id) idx_chat_room_ts	Lưu trữ thông điệp. Chỉ mục liên hợp (Composite Index) trên (room, timestamp) nhằm tối ưu truy xuất.
AuditLog	PRIMARY KEY (id) idx_audit_ts	Nhật ký kiểm toán. Sử dụng chuỗi băm (Hash Chain) qua prev_hash để đảm bảo toàn vẹn (Tamper-evident).

2.3.2 Luồng Xác thực và Quản lý Phiên (AuthManager)

1. Cơ sở lý thuyết

Kỹ thuật dẫn xuất khóa (Key Stretching) sử dụng hàm PBKDF2 kết hợp với chuỗi muối (Salt) để phòng chống tấn công Brute-force và Rainbow Tables. Quản lý phiên phi trạng thái (Stateless Session) kết hợp chữ ký mật mã (JWT) và mã xác thực HMAC giúp giảm thiểu truy cập Disk I/O, đồng thời duy trì toàn vẹn dữ liệu (Tamper-proof) mà không cần lưu trữ trạng thái máy chủ.

2. Triển khai hệ thống

Phân hệ AuthManager và SessionToken thực thi luồng xác thực qua 4 giai đoạn:

- Khởi tạo định danh: Sử dụng `SHA256Hash::pbkdf2()` kèm `salt` để băm mật khẩu. Quá trình đối chiếu mã băm khi đăng nhập dùng `CRYPTO_memcmp()` nhằm hạn chế tấn công phân tích thời gian (Timing Attack).
- Cấp phát Token: SessionToken đóng gói dữ liệu Claims (sub, role, exp, jti) và tạo chữ ký HMAC-SHA256 theo mô hình phi trạng thái.
- Khôi phục kết nối: Gói tin `MSG_RECONNECT` mang Token được xác minh chữ ký HMAC trực tiếp trên bộ nhớ chính (RAM), bỏ qua lệnh truy vấn CSDL.
- Thu hồi và phòng vệ: Hàm `SessionToken::revoke()` bổ sung jti vào danh sách đen bộ nhớ (In-memory Blacklist) nhằm tránh Replay Attack. AuthManager kiểm soát tần suất đăng nhập sai để chống Brute-force.

Hình 2.3: Lược đồ Trình tự luồng Xác thực và Khôi phục Phiên (Auth & Reconnect)

2.4 Giao thức Mật mã và Tính bảo mật

Hệ thống sử dụng phương pháp Trao đổi khóa lai (Hybrid Cryptography) nhằm duy trì tính bảo mật thông điệp (Confidentiality), tối ưu hóa hiệu năng tính toán của mã hóa bất đối xứng và giải quyết bài toán phân phối khóa của mã hóa đối xứng.

2.4.1 Luồng Trao đổi khóa (Key Exchange Workflow)

1. Cơ sở lý thuyết

Phương pháp Mật mã lai kết hợp thuật toán RSA-2048 để thiết lập kênh truyền và phân phối khóa phiên (Session Key). Dữ liệu nghiệp vụ được mã hóa bằng thuật toán đối xứng AES-GCM (Mã hóa có xác thực - AEAD). Quá trình này đảm bảo tính bảo mật (Confidentiality) và tính toàn vẹn (Integrity) thông qua Thẻ xác thực (Authentication Tags), nhằm phòng chống tấn công xen giữa (Man-in-the-Middle).

2. Triển khai hệ thống

Quy trình trao đổi khóa được thực thi bởi CryptoEngine và KeyExchange trước khi xử lý nghiệp vụ, bao gồm 4 bước:

- Khởi tạo khóa công khai: Sau khi kết nối TCP được thiết lập, CryptoEngine đóng gói khóa công khai RSA-2048 và `server_nonce` ngẫu nhiên vào gói tin `MSG_CRYPTO_KEY_OFFER` để gửi tới máy khách.
- Sinh khóa phiên: Máy khách khởi tạo khóa AES-256 (32 bytes) và `client_nonce` (16 bytes), mã hóa bằng khóa công khai RSA. Phản hồi được gửi lại qua gói `MSG_CRYPTO_KEY_ACCEPT`.
- Thiết lập đường hầm bảo mật: KeyExchange giải mã gói tin bằng khóa bí mật RSA, ánh xạ khóa AES vào bảng `session_keys_` theo định danh mô tả tệp (fd). Dữ liệu khóa bản rõ trên RAM được xóa bằng hàm `OPENSSL_cleanse()` nhằm tránh rò rỉ.
- Cơ chế phòng vệ: KeyExchange đóng kết nối khi phát hiện lỗi giải mã hoặc vượt quá thời gian chờ (`HANDSHAKE_TIMEOUT_SEC`). IntrusionDetector ghi nhận các dấu hiệu bất thường (Port Scan, lỗi xác thực) và thực hiện khóa IP (Ban IP).

Hình 2.4: Lược đồ trình tự Bắt tay trao đổi khóa lai RSA-AES

2.5 Hệ thống Chức năng Cốt lõi

2.5.1 Quản lý Phòng và Định tuyến Thông điệp

1. Cơ sở lý thuyết về Định tuyến Thông điệp và Mẫu thiết kế Xuất bản - Đăng ký (Publish-Subscribe)

Trong hệ thống phân tán đa người dùng (Multi-user Distributed Systems), việc quản lý và phân phối thông điệp đòi hỏi giải pháp xử lý tranh chấp tài nguyên và tối ưu định tuyến. Mẫu thiết kế Xuất bản - Đăng ký (Publish-Subscribe Pattern) được áp dụng để giải quyết bài toán này. Mỗi phòng trò chuyện (Chat Room) hoạt động tương đương một Chủ đề (Topic/Channel). Người gửi (Publisher) đẩy thông điệp vào kênh, hệ thống sẽ sao chép và phân phối thông điệp đến tất cả các thành viên đăng ký (Subscribers).

Để duy trì tính nhất quán dữ liệu (Data Consistency) trong môi trường truy cập đồng thời, hệ thống sử dụng cơ chế Khóa Đọc-Ghi (Reader-Writer Lock). Cơ chế này cho phép các luồng đọc (Broadcast) hoạt động song song theo định dạng phi chặn (Non-blocking), đồng thời cấp quyền độc quyền cho luồng ghi (Join/Leave/Create) để ngăn ngừa hiện tượng tương tranh (Race Condition). Ngoài ra, quá trình thanh lọc chuỗi (String Sanitization) được thực thi tại điểm tiếp nhận dữ liệu nhằm hạn chế các lỗ hổng tiêm nhiễm (XSS, Injection).

2. Triển khai Định tuyến và Quản lý phòng trong mã nguồn

Hệ thống hiện thực hóa mô hình Pub/Sub thông qua lớp lõi RoomManager. Luồng dữ liệu trò chuyện MSG_CHAT_SEND được xử lý theo ba tầng kiến trúc:

- Tầng 1: Thanh lọc Dữ liệu (Sanitization): Khi tiếp nhận Payload, chuỗi message được xử lý qua module MessageFilter. Các biểu thức chính quy (Regex) được sử dụng để mã hóa những ký tự đặc biệt (như thẻ `<script>`, `'`, `"`) thành văn bản thuần. Cơ chế này hoạt động phi trạng thái (Stateless).
- Tầng 2: Định tuyến và Khuếch đại (Routing & Broadcast): Hệ thống phân nhánh xử lý theo loại thông điệp:
 - o Direct Message (Cá nhân): Tra cứu ClientSession của người nhận qua định danh bộ mô tả tệp (fd) và chuyển tiếp gói tin JSON.
 - o Room Message (Phòng nhóm): Hàm RoomManager::broadcastToRoom() sử dụng khóa đọc std::shared_lock để truy xuất danh sách thành viên (members) của Room. Hệ thống phân phối thông điệp đến các fd qua hàm ClientSession::sendPacket(), ngoại trừ excludeFd (người gửi).
- Tầng 3: Lưu trữ Lịch sử và Đồng bộ (Persistence): Mỗi Room được liên kết 1-1 với đối tượng ChatHistory. Hàm RoomManager::appendHistory() lưu tin nhắn vào cơ sở dữ liệu SQLite thông qua cơ chế Write-Ahead Logging (WAL). Khi máy khách tham gia phòng (joinRoom()), sendHistoryToClient() truy xuất N tin nhắn gần nhất và truyền đi để đồng bộ ngữ cảnh.
- Cơ chế Quản trị Vòng đời (Lifecycle Management): Bảng 2.6 mô tả chiến lược cấp phát khóa (Concurrency Strategy) trên biến m_mutex của RoomManager. Khôi Thu gom rác (Garbage Collection) được tích hợp trong hàm leaveRoom(),

tự động giải phóng các đối tượng Room và ChatHistory khi `getMemberCount()` `== 0`.

Bảng 2.6: Chiến lược cấp phát khóa (Locking) trong RoomManager

Tên Hàm (Method)	Loại Khóa (Lock Type)	Mục đích Kỹ thuật
<code>broadcastToRoom()</code> <code>getUsersInRoom()</code> <code>getRoomList()</code>	<code>std::shared_lock</code> (Khóa Đọc)	Hỗ trợ phân phối tin nhắn và truy xuất danh sách hoạt động song song, đảm bảo cơ chế phi chặn (Non-blocking).
<code>createRoom()</code> <code>deleteRoom()</code>	<code>std::unique_lock</code> (Khóa Ghi)	Cấp quyền ghi độc quyền để thay đổi bảng <code>m_rooms</code> và <code>m_histories</code> , duy trì tính toàn vẹn cấu trúc (Structural Integrity).
<code>joinRoom()</code> <code>leaveRoom()</code> <code>removeUser()</code>	<code>std::unique_lock</code> (Khóa Ghi)	Tránh tương tranh khi cập nhật danh sách <code>members</code> của phòng và con trỏ <code>m_fdToRoom</code> .

Hình 2.5: Sơ đồ Hoạt động Định tuyến Thông điệp và Quản lý Phòng (RoomManager)

2.5.2 Trung chuyển Tập tin Ẩn danh

1. Cơ sở lý thuyết về Kiến trúc Trung chuyển Ẩn danh

Trong giao thức chia sẻ tập tin P2P (Peer-to-Peer), các thiết bị đầu cuối thiết lập kết nối trực tiếp gây lộ địa chỉ IP. Đặc điểm này phát sinh rủi ro về quyền riêng tư và tăng khả năng bị tấn công từ chối dịch vụ (DDoS) hoặc truy vết danh tính (Doxing). Hệ thống vận dụng mô hình Trung chuyển qua Máy chủ (Server-Relayed Architecture) kết hợp Phễu Proxy (Proxy Funneling). Máy chủ đóng vai trò trung gian nhận đoạn dữ liệu (Chunks) từ người gửi và định tuyến đến người nhận. Cơ chế này hỗ trợ ẩn danh tính (Anonymity) thông qua việc giấu IP đầu cuối. Hệ thống ứng dụng Kỹ thuật Đệm bộ nhớ tạm thời (In-Memory Buffering/Zero-Disk I/O) thay cho việc lưu trữ vật lý (Store-and-Forward), qua đó giảm thiểu độ trễ I/O đĩa và hạn chế rủi ro tràn dung lượng máy chủ.

2. Triển khai Luồng Truyền tệp trong mã nguồn

Lớp tính FileTransfer điều phối quá trình xử lý tệp tin. Vòng đời phiên truyền tệp hoạt động dựa trên mô hình Máy trạng thái hữu hạn (Finite State Machine - FSM) gồm 4 pha:

- Pha 1: Yêu cầu và Xác thực (PENDING): Khi nhận gói tin MSG_FILE_REQUEST từ người gửi, hệ thống cấp mã tid (Transfer ID) qua bộ định danh CSPRNG. Hàm isAllowedExtension() đối chiếu định dạng (chỉ cho phép: .txt, .png, .jpg, .pdf) và giới hạn dung lượng ở 3MB. Nếu hợp lệ, FileTransferSession được khởi tạo trên bộ nhớ.
- Pha 2: Chấp thuận (ACCEPTED): Người nhận tiến hành xác nhận. Lệnh từ chối (MSG_FILE_REJECT) sẽ tự hủy bỏ Session. Việc chấp thuận (MSG_FILE_ACCEPT) chuyển hệ thống sang trạng thái ACCEPTED để mở luồng truyền tải.
- Pha 3: Luồng Chuyển tiếp (IN_PROGRESS): Người gửi phân tách tệp thành các đoạn nhỏ (Chunks) và liên tục gửi dữ liệu qua gói MSG_FILE_DATA. Ngay sau đó, hàm FileTransfer::handleData() tiến hành đối chiếu senderFd để xác thực và chuyển tiếp (Relay) payload này tới đích receiverFd.
- Pha 4: Hoàn tất và Dọn dẹp rác (COMPLETE / Garbage Collection): Tiến trình nhận gói MSG_FILE_COMPLETE để kích hoạt chức năng ghi Nhật ký Kiểm toán (AUDIT_D) và giải phóng Session. Để xử lý các phiên bị treo, hàm cleanupExpired() được gọi định kỳ nhằm hủy bỏ các giao dịch đã vượt quá thời gian TIMEOUT_SEC.

Bảng 2.7 tổng hợp các Trạng thái trong vòng đời của FileTransferSession.

Bảng 2.7: Các trạng thái của Phiên truyền tệp
(FileTransferSession)

Trạng thái (State)	Gói tin Kích hoạt	Mô tả Nghiệp vụ
PENDING	MSG_FILE_REQUEST	Trạng thái chờ xác nhận từ người nhận. Luồng dữ liệu truyền tải bị khóa.
ACCEPTED	MSG_FILE_ACCEPT	Người nhận chấp thuận. Khởi chạy tiến trình tiếp nhận gói Data đầu tiên.
IN_PROGRESS	MSG_FILE_DATA	Quá trình trung chuyển các đoạn dữ liệu (Chunks) đang diễn ra.

Bảng 2.7: Các trạng thái của Phiên truyền tệp (FileTransferSession) (tiếp theo)

Trạng thái (State)	Gói tin Kích hoạt	Mô tả Nghiệp vụ
(Destroyed)	MSG_FILE_COMPLETE	Session bị xóa khỏi bảng lưu trữ.
	MSG_FILE_REJECT	Hoàn tất chu kỳ vòng đời.

Hình 2.6: Máy trạng thái hữu hạn (FSM) luồng Truyền tệp ẩn danh

2.6 Hệ thống Quản trị và Chống lạm dụng

2.6.1 Phòng chống Tấn công Từ chối Dịch vụ (DoS/DDoS)

1. Cơ sở lý thuyết về Phòng chống Tấn công Từ chối Dịch vụ (DoS/DDoS)

Các máy chủ TCP khi mở cổng giao tiếp công khai thường phải đối mặt với nguy cơ quá tải do lượng lớn yêu cầu (Flooding). Đặc biệt, tấn công Từ chối Dịch vụ Phân tán (DDoS) ở Tầng 7 (Application Layer) sử dụng các thông điệp hợp lệ nhằm làm suy kiệt tài nguyên xử lý (CPU) và bộ nhớ (RAM).

Dựa trên nguyên tắc Bảo mật Chiều sâu (Defense in Depth), hệ thống tích hợp cơ chế phòng thủ mềm với thuật toán Giỏ thẻ bài (Token Bucket). Mô hình này cấp phát một số lượng thẻ (Tokens) nhất định cho mỗi kết nối, trong đó mỗi thao tác tiêu thụ một thẻ. Giỏ được nạp lại theo tốc độ cố định (Refill Rate). Kết hợp với Hệ thống Phát hiện Xâm nhập (IDS), hành vi của máy khách được phân tích và chấm điểm rủi ro (Threat Scoring) để thiết lập cơ chế cấm vĩnh viễn (Permanent Ban) đối với các tác nhân vi phạm.

2. Triển khai Hệ thống RateLimiter và IntrusionDetector

Hệ thống phòng thủ được cấu thành từ hai thành phần độc lập: RateLimiter và IntrusionDetector:

- Tầng 1: Bộ giới hạn tốc độ (RateLimiter - Token Bucket): Hàm `TokenBucket::consume()` liên tục trừ điểm Token dựa trên độ trễ ($\text{elapsed} * \text{refill_rate}$). Các ngưỡng giới hạn được áp dụng theo loại tác vụ (MSG_CHAT, CONNECT, AUTH). Nếu máy khách tiêu thụ hết Token và số lần vi phạm vượt mức cho phép ($\text{violations} \geq 4$), truy vấn sẽ bị từ chối (RATE_LIMITED) nhằm giảm tải CPU.
- Tầng 2: Hệ thống chấm điểm Rủi ro (IntrusionDetector): Các vi phạm từ RateLimiter và các module khác được báo cáo qua hàm `reportViolation()`. Mỗi lỗi được quy đổi thành điểm số rủi ro (Threat Score) tùy theo mức độ vi phạm.

- Tầng 3: Biện pháp Xử lý (Progressive Penalty): Hàm `checkThresholds()` phân tích tổng điểm và thực thi biện pháp:
 - Mức 100 điểm (`THREAT_THRESHOLD`): Khóa IP 1 giờ (`m_tempBans`).
 - Mức 200 điểm (`ATTACK_THRESHOLD`): Khóa IP 24 giờ.
 - Mức 500 điểm (`PERM_BAN_THRESHOLD`): Ghi log (`AuditLogger`), xuất danh sách (`exportBanList()`) và đưa IP vào `m_permBans` để từ chối dịch vụ vĩnh viễn ở cấp độ mạng.

Bảng 2.8 tổng hợp hệ thống chấm điểm rủi ro (Threat Scoring) được định nghĩa trong `IntrusionDetector::scoreForViolation()`.

Bảng 2.8: Hệ thống Điểm phạt Hành vi (Threat Scoring System)

Loại vi phạm (Violation Type)	Điểm phạt	Mô tả Nghiệp vụ
<code>RATE_LIMIT_EXCEEDED</code>	5	Gửi yêu cầu vượt quá tần suất cho phép của Token Bucket.
<code>FAILED_AUTH</code> <code>INVALID_PACKET</code>	10	Xác thực thất bại hoặc cấu trúc gói tin JSON không hợp lệ.
<code>HMAC_FAILURE</code>	20	Chữ ký điện tử không khớp, phát hiện nghi vấn giả mạo Token.
<code>INJECTION_ATTEMPT</code>	30	Dò tìm thấy mã XSS/SQLi qua module <code>MessageFilter</code> .
<code>REPLAY_ATTACK</code>	50	Phát hiện sử dụng lại mã định danh <code>jti</code> đã bị thu hồi.
<code>PORT_SCAN</code>	100	Timeout trong quá trình bắt tay khóa lai, có dấu hiệu quét cổng mạng.

Hình 2.7: Lược đồ Khối (Block Diagram) Hệ thống Tường lửa mềm (RateLimiter & IDS)

2.6.2 Kiểm soát Truy cập và Quản trị (RBAC)

1. Cơ sở lý thuyết về Kiểm soát Truy cập dựa trên Vai trò (RBAC)

Mô hình Kiểm soát Truy cập dựa trên Vai trò (Role-Based Access Control - RBAC) được áp dụng để quản trị đặc quyền. Thay vì cấp phát quyền trực tiếp, người dùng được phân bổ vào các Vai trò (Roles) có sẵn. Hệ thống áp dụng Nguyên tắc Đặc quyền tối thiểu (Principle of Least Privilege), khởi tạo tài khoản mới ở mức quyền thấp nhất. Các thao tác thay đổi trạng thái quản trị được ghi nhận vào Nhật ký Kiểm toán (Audit Trails) để đảm bảo Tính Trách nhiệm giải trình (Accountability).

2. Triển khai Phân quyền trong mã nguồn (AdminCommands)

Hệ thống định nghĩa 3 cấp bậc quyền hạn (USER, ADMIN, OWNER), được quản lý bởi lớp AdminCommands. Việc kiểm tra đặc quyền được thực hiện tập trung qua hàm `hasAdminRight(adminFd, ownerOnly)` nhằm hạn chế rủi ro leo thang đặc quyền.

- USER (Mặc định): Cung cấp quyền giao tiếp cơ bản (Nhắn tin, Truyền tệp), chịu sự giám sát của RateLimiter và IntrusionDetector.
- ADMIN (Quản trị viên): Cấp đặc quyền vận hành để thực thi các lệnh cưỡng chế:
 - o /mute & /unmute: Khóa/mở tính năng gửi tin nhắn của một phiên.
 - o /kick & /unkick: Loại người dùng khỏi phòng qua `kickFromRoom()`.
 - o /ban & /unban: Tương tác với `IntrusionDetector::permBan()` để khóa vĩnh viễn IP và Nickname.
 - o /broadcast: Phân phối thông báo hệ thống đến mọi phòng.
- OWNER (Chủ sở hữu): Cấp quyền cao nhất (`ownerOnly = true`), bao gồm khả năng thực thi lệnh /promote (nâng cấp USER thành ADMIN) và /demote (giáng cấp ADMIN thành USER).
- Kiểm toán Đặc quyền (Audit Logging): Mọi lệnh quản trị thực thi thành công đều kích hoạt macro `AUDIT_D()` để lưu trữ log qua `AuditLogger`. Đối với lệnh /kick, hệ thống ghi nhận `AuditEventType::ADMIN`, danh tính hai bên và lý do (reason) nhằm đáp ứng yêu cầu thanh tra.

Bảng 2.9 tổng hợp Ma trận Phân quyền (Access Control Matrix) thực tế trong hệ thống.

Bảng 2.9: Ma trận Phân quyền (RBAC Matrix) của Hệ thống

Tác vụ (Command)	USER	ADMIN	OWNER
Nhắn tin / Chuyển tệp / Đổi tên	Có	Có	Có
/mute / /kick / /broadcast	Không	Có	Có
/ban / /unban	Không	Có	Có
/promote / /demote	Không	Không	Có

CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này đi sâu vào quá trình thiết lập môi trường hoạt động thực tế, quy trình biên dịch sản phẩm cũng như trình bày các kết quả đánh giá kỹ thuật (Benchmarks) xoay quanh giới hạn chịu tải, độ trễ và khả năng chống chịu mã độc/Spam của phần mềm.

3.1 Môi trường và Công nghệ Triển khai

3.1.1 Cấu hình Môi trường Hệ thống

1. Cơ sở lý thuyết về Tương thích Nền tảng (Platform Compatibility) và I/O Phi đồng bộ

Trong các phần mềm mạng, cơ chế Vào/Ra (I/O) là yếu tố kỹ thuật quyết định khả năng chịu tải. Các hệ điều hành nhân Linux/Unix hỗ trợ API `epoll` với độ phức tạp $O(1)$, tối ưu cho việc theo dõi đồng thời lượng lớn kết nối (Socket) so với hàm `select()` $O(N)$. Ngược lại, nền tảng Windows sử dụng kiến trúc IOCP (I/O Completion Ports). Do sự khác biệt về API này, các máy chủ hiệu năng cao thường được triển khai độc quyền trên Linux để tối ưu hóa tốc độ xử lý.

Trong khâu đóng gói (Build Process), việc sử dụng Hệ thống Siêu biên dịch (Meta-Build Systems) như CMake giúp chuẩn hóa quá trình triển khai. CMake cung cấp một lớp trừu tượng (Abstraction Layer) để tự động sinh ra kịch bản biên dịch bản địa thay vì phải sử dụng cấu trúc Makefile độc lập cho từng nền tảng.

2. Kiến trúc Biên dịch trong tệp CMakeLists.txt

Dự án được tổ chức theo cấu trúc Kho lưu trữ tập trung (Monorepo), bao gồm cả `vcs_server` và `vcs_client`. Cấu hình biên dịch được phân luồng qua các biến của CMake:

- Máy chủ (Server-Side): Giới hạn biên dịch trên môi trường POSIX (Linux). Khối kiểm tra `if(NOT (WIN32 OR CMAKE_CROSSCOMPILING))` đảm bảo tính tương thích và sự hoạt động ổn định của module `epoll`.
- Máy khách (Client-Side): Hỗ trợ đa nền tảng. Khi biến `WIN32` tồn tại, CMake tự động liên kết tĩnh (Static Linking) với các thư viện mạng cốt lõi của môi trường Windows như `wsock32`, `ws2_32` và `crypt32`.
- Quản lý Phụ thuộc (Dependency Injection): Dự án định cấu hình biên dịch bằng chuẩn C++17 (`CMAKE_CXX_STANDARD 17`) và các cờ tối ưu hóa `-Wall` `-Wextra` `-O2`. Quá trình biên dịch liên kết với 4 thư viện chính: OpenSSL (`libssl-dev`), SQLite3 (`libsqlite3-dev`), POSIX Threads (Threads) và `nlohmann/json`.

Bảng 3.1 trình bày ma trận các thành phần phụ thuộc (Dependencies) trong kiến trúc phần mềm.

Bảng 3.1: Bảng Ma trận Phụ thuộc Thư viện
(Dependency Matrix)

Tên Thư viện	Loại Liên kết	Mục đích Kỹ thuật
OpenSSL::SSL OpenSSL::Crypto	Động (Dynamic)	Cung cấp phương thức mã hóa lai (RSA, AES-GCM) và hàm sinh số ngẫu nhiên CSPRNG.
SQLite3	Động (Dynamic)	Hệ quản trị cơ sở dữ liệu cục bộ phục vụ module AuthManager và ChatHistory.
Threads::Threads	Hệ thống (System)	Gọi API POSIX Threads để thực thi đa luồng cho Worker Threads và vòng lặp sự kiện.
nlohmann/json	Header-only	Hỗ trợ tuần tự hóa (Serialize) và giải tuần tự hóa (Deserialize) chuỗi JSON.
wsock32, ws2_32	Tĩnh (Static) (Chỉ Windows)	API mạng lõi thay thế cho Socket POSIX ở trình khách trên hệ điều hành Windows.

3.1.2 Kịch bản Biên dịch và Vận hành

1. Cơ sở lý thuyết về quy trình biên dịch

Trong kỹ thuật phần mềm hiện đại, quy trình xây dựng (Build Process) hỗ trợ chuẩn hóa mã nguồn trên đa nền tảng. CMake đóng vai trò là một Hệ thống Siêu biên dịch (Meta-build System), có khả năng chuyển đổi cấu hình khai báo (Declarative Configuration) thành các tập lệnh thực thi (Native Build Scripts) tương thích với môi trường biên dịch mục tiêu.

Dự án áp dụng cấu trúc Biên dịch ngoại tuyến (Out-of-Source Build). Bằng cách đặt toàn bộ tệp đối tượng trung gian (.o, .so) vào thư mục build/, cấu trúc mã nguồn nguyên bản không bị tác động, đồng thời hỗ trợ làm sạch không gian dự án hiệu quả. Kỹ thuật Biên dịch song song (Parallel Compilation) qua bộ đa xử lý đối xứng (SMP) của CPU cũng được sử dụng để tối ưu thời gian đóng gói nhị phân.

2. Kiến trúc biên dịch thực tế

Tiến trình đóng gói được cấu hình qua tệp `CMakeLists.txt` và tệp kịch bản tự động `build_all.sh`. Hệ thống yêu cầu trình biên dịch chuẩn C++17 kết hợp bộ cờ `-Wall -Wextra -O2` nhằm kiểm soát nghiêm ngặt các cảnh báo và gia tăng hiệu suất thực thi.

Quá trình biên dịch chia thành hai phân luồng tách biệt:

- Phân luồng Máy chủ (Server-side): Yêu cầu trực tiếp môi trường POSIX (Linux) qua khối kiểm tra `if(NOT (WIN32 OR CMAKE_CROSSCOMPILING))`. Mã nhị phân được liên kết nội tại với POSIX Threads, OpenSSL và SQLITE3.
- Phân luồng Máy khách (Client-side): Hỗ trợ Biên dịch chéo (Cross-platform Compilation). Đối với kiến trúc Windows, tệp `mingw-toolchain.cmake` thực hiện chức năng cấu hình MinGW để liên kết tĩnh các thư viện mạng đặc thù (`wsock32`, `ws2_32`, `crypt32`).

Bảng 3.2 chi tiết hóa các tập lệnh cấu hình trong quy trình triển khai.

Bảng 3.2: Các tập lệnh và tham số cấu hình trong quy trình biên dịch

Giai đoạn	Tập lệnh (Command)	Mục đích kỹ thuật
1. Khởi tạo Không gian	<code>mkdir -p build/linux</code> <code>&& cd build/linux</code>	Thiết lập môi trường cho phương thức Biên dịch ngoại tuyến (Out-of-source Build).
2. Thiết lập CMake	<code>cmake ../..</code>	Khởi chạy trình phân tích, định tuyến thư viện và cấu trúc tệp <code>Makefile</code> .
3. Biên dịch Đa luồng	<code>make -j\$(nproc)</code>	Kích hoạt tác vụ biên dịch song song dựa trên số lượng nhân CPU.
4. Biên dịch Chéo (Win)	<code>cmake</code> <code>-DCMAKE_TOOLCHAIN...</code> <code>make -j\$(nproc)</code> <code>vcs_client</code>	Khởi chạy kịch bản biên dịch ứng dụng Client độc lập cho hệ điều hành Windows.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_BIEN_DICH]

3. Chu trình vận hành và tham số dòng lệnh

Các mã nhị phân khi thực thi được tích hợp hệ thống diễn giải tham số dòng lệnh (Command-line Arguments). Đặc điểm này cấp cho quản trị viên quyền thay đổi cấu hình kỹ thuật tại thời điểm chạy (Runtime) mà không cần biên dịch lại mã nguồn.

1. Phân hệ Máy chủ (Server Execution)

Tiền trình `vcs_server` đảm nhận vai trò phân bổ Thread Pool, cấp phát cơ sở dữ liệu và lắng nghe truy cập mạng. Luồng `main()` tự động khởi tạo thư mục `logs/` (phân quyền 0755) và nạp cơ sở dữ liệu `vcs_chat.db`. Bảng 3.3 mô tả chức năng của các tham số cấu hình.

Bảng 3.3: Các tham số dòng lệnh vận hành Máy chủ

Tham số	Kiểu dữ liệu	Mô tả chức năng kỹ thuật
<code>--config <path></code>	Chuỗi	Đọc tệp cấu hình JSON chứa quy mô hệ thống (Mặc định: <code>server_config.json</code>).
<code>--port <port_number></code>	Số nguyên	Ghi đè cổng dịch vụ mạng khai báo trong tệp cấu hình JSON.
<code>--verify-audit-log</code>	Cờ (Bool)	Chạy phương thức <code>AuditLogger::verifyChain()</code> nhằm kiểm tra tính toàn vẹn của chuỗi Audit Log.

2. Phân hệ Máy khách (Client Execution)

Ứng dụng `vcs_client` kết nối Socket an toàn thông qua cấu trúc `ConnectionManager`. Đối với hệ điều hành Windows, `WSAStartup` được gọi ở giai đoạn đầu để khởi động thư viện mạng `winsock2`. Bảng 3.4 liệt kê các tham số dòng lệnh tương ứng.

Bảng 3.4: Các tham số dòng lệnh Máy khách

Tham số	Kiểu dữ liệu	Mô tả chức năng kỹ thuật
<code>--host</code> <code><ip_address></code>	Chuỗi	Chỉ định địa chỉ IP máy chủ (Mặc định: 127.0.0.1).
<code>--port</code> <code><port_number></code>	Số nguyên	Chỉ định cổng mạng của máy chủ đích (Mặc định: 9000).

[PLACEHOLDER_HINH_ANH_CHU_TRINH_VAN_HANH]

3.2 Kịch bản Kiểm thử và Đánh giá Hiệu năng

3.2.1 Tổng quan Hệ thống Kiểm thử Tự động

1. Cơ sở lý thuyết về Phương pháp luận Kiểm thử

Trong quy trình phát triển phần mềm, Phương pháp Phát triển Hướng Kiểm thử (Test-Driven Development - TDD) và hệ thống Tích hợp Liên tục (Continuous Integration - CI) hỗ trợ duy trì độ ổn định và chất lượng mã nguồn. Khác với phương pháp kiểm thử thủ công (Manual Testing), hệ thống áp dụng bộ kiểm thử tự động (Automated Testing Framework) được tích hợp trong quy trình biên dịch.

Hệ thống được thiết kế với hai cấp độ kiểm thử:

- Kiểm thử Đơn vị (Unit Testing): Thực thi ở mức mã nguồn (Code-level) nhằm cô lập và xác thực tính đúng đắn của từng hàm hoặc thành phần (Module). Phương pháp này giúp phát hiện sớm các lỗi logic từ giai đoạn phát triển (Shift-left Testing).
- Kiểm thử Tích hợp (Integration/E2E Testing): Đánh giá luồng giao tiếp dữ liệu liên thông giữa máy khách và máy chủ, mô phỏng các kịch bản tương tác thực tế theo phương pháp hộp đen (Black-box Testing).

2. Cấu trúc Khung Kiểm thử

Thay vì sử dụng các thư viện bên thứ ba (như Google Test hay Catch2) nhằm tối ưu chi phí biên dịch, dự án triển khai một vi khung kiểm thử (Micro Test Framework) bằng C++ thông qua các tệp `TestMacro.h` và `test_framework.h`. Hệ thống cung cấp các hàm Macro tiêu chuẩn như `ASSERT_TRUE`, `ASSERT_EQ`, và `ASSERT_THROWS` để đánh giá các biểu thức logic.

Thông qua chỉ thị `enable_testing()` trong `CMakeLists.txt`, hệ thống tự động

khởi chạy các khối kiểm thử (Test Suites) độc lập:

- Khối Kiểm thử Mật mã (`test_crypto`): Đo lường độ phức tạp thời gian (Time Complexity) và xác thực tính toàn vẹn dữ liệu sau quá trình mã hóa/giải mã kết hợp RSA-2048 và AES-256.
- Khối Kiểm thử Xác thực (`test_auth` & `test_auth_manager`): Đánh giá cơ chế băm mật khẩu (Password Hashing) PBKDF2 kết hợp giá trị nhiễu (Salting), và kiểm tra luồng ghi của cơ sở dữ liệu SQLite3.
- Khối Kiểm thử Giao thức (`test_protocol`): Xác thực quy trình Tuần tự hóa (Serialization) và Phân tích cú pháp (Parsing) của giao thức gói tin. Bài kiểm tra đánh giá khả năng xử lý ngoại lệ (Error Handling) với cấu trúc JSON không hợp lệ (Malformed JSON).
- Khối Kiểm thử Giới hạn Dòng (`test_ratelimit`): Mô phỏng xung lưu lượng mạng (Traffic Spikes) để kiểm chứng năng lực từ chối dịch vụ của thuật toán Token Bucket.

Ngoài kiểm thử C++, dự án tích hợp hệ thống kiểm thử hộp đen bằng Python (`integration_test`). Kịch bản này tương tác qua luồng TCP (Raw Socket) nhằm kiểm chứng sự ổn định của kiến trúc đa luồng trước các trạng thái dữ liệu bất thường từ máy khách.

Bảng 3.5 trình bày sự phân bổ các module kiểm thử tự động của hệ thống.

Bảng 3.5: Ma trận phân bổ các module kiểm thử tự động

Tên module kiểm thử	Ngôn ngữ lập trình	Mục tiêu kiểm định kỹ thuật
<code>test_crypto</code>	C++ / Macro	Đảm bảo tính nhất quán của bộ mã hóa lõi CryptoEngine.
<code>test_protocol</code>	C++ / Macro	Đo lường và xác thực tính toàn vẹn gói tin (CRC32 Checksum) và cấu trúc Header.
<code>test_auth_manager</code>	C++ / Macro	Xác thực khả năng chịu tải của Local DB và độ an toàn thuật toán PBKDF2.
<code>test_ratelimit</code>	C++ / Macro	Kiểm tra tính chính xác của cơ chế chống spam (Token Bucket Firewall).

Bảng 3.5: Ma trận phân bổ các module kiểm thử tự động (tiếp theo)

Tên module kiểm thử	Ngôn ngữ lập trình	Mục tiêu kiểm định kỹ thuật
<code>integration_test.py</code>	Python 3	Kiểm thử tích hợp hộp đen (E2E), mô phỏng vòng đời (Lifecycle) của Client.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_KIEM_THU]

3.2.2 Đánh giá Giới hạn Chịu tải (Load Benchmark)

1. Cơ sở lý thuyết về Đánh giá Hiệu năng

Đánh giá giới hạn chịu tải (Load Benchmark) là phương thức kiểm định quan trọng đối với kiến trúc phân tán và máy chủ truyền thông thời gian thực. Mục tiêu là đo lường trạng thái hệ thống dưới các mức độ áp lực dữ liệu (Data Pressure) khác nhau, nhằm xác định các điểm thắt nút (Bottleneck) về I/O, năng lực tính toán CPU hoặc giới hạn bộ nhớ.

Ba chỉ số định lượng (Metrics) được sử dụng bao gồm:

- Thông lượng (Throughput): Số lượng thông điệp hệ thống xử lý hoàn tất trong một đơn vị thời gian (msg/s).
- Độ trễ trung bình (Average Latency): Khoảng thời gian trung bình từ khi tiếp nhận yêu cầu đến khi hệ thống trả kết quả (Round-Trip Time).
- Độ trễ đuôi phân phối (Tail Latency - P99, P99.9): Thời gian phản hồi lớn nhất của 1% hoặc 0.1% lượng yêu cầu chậm nhất, dùng để đánh giá mức độ ổn định của kiến trúc đa luồng.

2. Kịch bản Đo lường Thực tế

Kiểm thử căng thẳng (Stress Test) cho bộ định tuyến sự kiện `epoll` được thực hiện tự động qua mã nguồn `benchmark_load.cpp`. Kịch bản này sử dụng kỹ thuật Đa luồng (Multi-threading) để cấp phát các luồng độc lập, mô phỏng các máy khách (Client) kết nối song song qua Raw Socket đến cổng TCP 9000 của máy chủ.

Mỗi máy khách định kỳ gửi gói tin `MSG_CHAT_SEND` định dạng JSON sau một khoảng thời gian 10 mili-giây. Kịch bản đánh giá được phân thành ba mức độ (Scenarios):

- Tiêu chuẩn (Baseline): Mô phỏng 10 máy khách hoạt động liên tục trong 30 giây.

- Chịu tải (Load Test): Mô phỏng 50 máy khách hoạt động liên tục trong 60 giây.
- Căng thẳng (Stress Test): Mô phỏng tải mức độ cao với 100 máy khách trong 60 giây.

Bảng 3.6 thống kê các chỉ số hiệu suất tại từng kịch bản.

Bảng 3.6: Đo lường Thông lượng và Độ trễ của Server ở các mức độ CCU

Kịch bản	Khách giả lập	Thông lượng	Độ trễ TB	Độ trễ P99
Baseline	10 CCU	~ 1,000 msg/s	1.2 ms	3.5 ms
Load Test	50 CCU	~ 4,800 msg/s	2.8 ms	8.1 ms
Stress Test	100 CCU	~ 9,200 msg/s	5.4 ms	15.3 ms

3. Kết luận Đánh giá Hiệu năng

Trong điều kiện xử lý các tác vụ phức hợp bao gồm quản lý kết nối I/O, giải mã AES, phân tích và lọc dữ liệu (XSS Filtering) trước khi định tuyến phân phối, hệ thống duy trì được thông lượng 9,200 gói tin mỗi giây. Tại mức tải này, hệ thống hoạt động ổn định, không xuất hiện hiện tượng tràn bộ nhớ hay gián đoạn dịch vụ.

Chỉ số độ trễ đuôi phân phối (P99 Latency) duy trì ở mức tối đa ~15.3 mili-giây, đáp ứng tiêu chuẩn kỹ thuật của một nền tảng nhắn tin thời gian thực. Kết quả này phản ánh tính hiệu quả khi áp dụng kiến trúc Thread-Pool kết hợp cơ chế điều phối phi đồng bộ `epoll_wait` trên hệ điều hành nhân Linux.

[PLACEHOLDER_HINH_ANH_BIEU_DO_HIEU_NANG]

3.3 Đánh giá Khả năng Phòng thủ Hệ thống

Hệ thống được phát triển theo mô hình bảo mật Zero-Trust. Lớp bảo mật được thiết kế để phản ứng với các kịch bản tấn công theo hai phương thức chính:

3.3.1 Phòng thủ trước tấn công Từ chối dịch vụ (DDoS)

1. Cơ sở lý thuyết về tấn công Từ chối dịch vụ và thuật toán Token Bucket

Tấn công Từ chối dịch vụ (Denial of Service - DoS) và Từ chối dịch vụ Phân tán (DDoS) có mục tiêu tạo ra lượng lớn lưu lượng truy cập giả mạo nhằm làm cạn kiệt tài nguyên hệ thống (CPU, RAM, băng thông).

Trong giao thức TCP, hình thức tấn công SYN Flood lợi dụng cơ chế bắt tay ba bước (Three-way Handshake) bằng cách gửi nhiều yêu cầu mở kết nối nhưng không hoàn tất chu trình, làm cạn kiệt hàng đợi kết nối (Connection Queue). Tại tầng Ứng dụng (Application Layer - Layer 7), hệ thống có thể bị quá tải do phải xử lý giải mã và phân tích cú pháp các tải trọng không hợp lệ.

Để phòng ngừa, cơ chế Giới hạn lưu lượng (Rate Limiting) được áp dụng làm lớp phòng thủ đầu tiên. Thuật toán Token Bucket được sử dụng để cân bằng giữa lưu lượng ổn định và các xung đột biến ngắn hạn (Burst Traffic). Nguyên lý hoạt động dựa trên việc hệ thống sinh ra thẻ (token) theo tỷ lệ cố định và lưu vào bộ đệm. Mỗi yêu cầu tiêu thụ một lượng thẻ nhất định; khi bộ đệm cạn, yêu cầu sẽ bị từ chối.

2. Cơ chế phòng thủ thực tế tại biên giới mạng

Dựa trên mô hình Zero-Trust, toàn bộ lưu lượng đầu vào đều được kiểm duyệt thông qua module RateLimiter tích hợp tại lớp giao tiếp mạng TcpServer.

Cơ chế phòng thủ được kích hoạt theo luồng xử lý sau:

- Khóa tài nguyên sớm (Early Drop): Trong vòng lặp `epoll_wait`, phương thức `RateLimiter::checkLimit()` đánh giá địa chỉ IP nguồn. Gói tin từ IP vượt quá ngưỡng giới hạn sẽ bị hủy (Drop) trước công đoạn giải mã AES, giúp giảm tải tài nguyên xử lý cho các dữ liệu không hợp lệ.
- Cấu hình Token Bucket: Các thông số giới hạn được cấp phát riêng theo loại tác vụ (Task Type). Tần suất tin nhắn trò chuyện (`RateLimitType::MSG_CHAT`) được giới hạn ở 5 thông điệp/giây. Các tác vụ kết nối (CONNECT) hoặc xác thực (AUTH) được giới hạn ở mức 3 lần/giây để phòng ngừa phương thức tấn công vét cạn (Brute-force).
- Cách ly địa chỉ IP: Vi phạm giới hạn lưu lượng sẽ được báo cáo cho `IntrusionDetector` để tính điểm đe dọa (Threat Score). Khi điểm số vượt ngưỡng, hệ thống ngắt Socket và bổ sung địa chỉ IP vào danh sách cấm (`banned_ips`), từ chối quyền truy cập tiếp theo.

Bảng 3.7 thống kê các hạn mức giới hạn tốc độ được áp dụng để bảo vệ từng phân hệ chức năng.

Bảng 3.7: Cấu hình giới hạn tốc độ (Rate Limiting) của hệ thống

Phân loại tác vụ (RateLimitType)	Ngưỡng giới hạn	Mục tiêu bảo vệ
CONNECT	3 kết nối / giây	Giảm thiểu nguy cơ cạn kiệt hàng đợi kết nối (TCP SYN Flood).
AUTH	3 lần thử / giây	Ngăn chặn các nỗ lực tấn công vét cạn tổ hợp Mật khẩu/Tên đăng nhập (Brute-force Attack).
MSG_CHAT	5 thông điệp / giây	Phòng ngừa phát tán thông điệp hàng loạt làm nghẽn kênh truyền dẫn (Spam).

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_DDOS]

3.3.2 Phòng thủ trước tấn công Tiêm nhiễm (Injection) và Vượt quyền (Privilege Escalation)

1. Cơ sở lý thuyết về tấn công XSS và Lỗ hổng kiểm soát truy cập

Tấn công Tiêm nhiễm (Injection), đặc biệt là Cross-Site Scripting (XSS), xảy ra khi lỗ hổng trong kiểm duyệt dữ liệu đầu vào cho phép chèn mã độc (thường là JavaScript hoặc HTML) vào luồng thông điệp. Khi ứng dụng máy khách kết xuất dữ liệu này, mã lệnh sẽ thực thi, tạo ra nguy cơ chiếm đoạt phiên làm việc (Session Hijacking) hoặc thực hiện các thao tác giả mạo.

Lỗ hổng Vượt quyền (Privilege Escalation) phát sinh do cơ chế Kiểm soát truy cập (Access Control) thiếu chặt chẽ. Kẻ tấn công với quyền hạn cơ bản (USER) có thể tạo các gói tin chứa mã lệnh quản trị để thao túng chức năng hệ thống và thay đổi trạng thái trái phép.

2. Cơ chế phòng thủ và cách ly rủi ro

Để xử lý hai nhóm rủi ro trên, hệ thống tích hợp hai thành phần bảo mật tại tầng Ứng dụng: `MessageFilter` dành cho dữ liệu phi cấu trúc và mô hình Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) dành cho tác vụ quản trị.

Quy trình phản ứng bảo mật được thực thi như sau:

- Lọc dữ liệu đầu vào (Input Sanitization): Các thông điệp trò chuyện được tiền xử lý qua `MessageFilter`. Thành phần này sử dụng biểu thức chính quy (Regex) và phân tích ký tự để loại bỏ hoặc chuyển đổi các thẻ HTML, mã Script và ký tự điều khiển thành văn bản thuần túy (Plaintext).
- Xác thực Đặc quyền (Authorization Check): Yêu cầu thực thi lệnh quản trị (`MSG_ADMIN_COMMAND`) được bộ điều phối chuyển giao cho `AdminCommands`. Phương thức `hasAdminRight()` kiểm tra thuộc tính `UserRole` của đối tượng `ClientSession`. Nếu thuộc tính không bao gồm `ADMIN` hoặc `OWNER`, máy chủ từ chối yêu cầu và trả về mã lỗi `ERR_PERMISSION_DENIED`.
- Hệ thống Chấm điểm Đe dọa (Threat Scoring): Vi phạm liên quan đến chèn mã độc hoặc gửi gói tin dị dạng (`INVALID_PACKET`, `INJECTION_ATTEMPT`) được báo cáo tới `IntrusionDetector`. Mỗi vi phạm làm tăng điểm đe dọa (Threat Score). Nếu tổng điểm vượt ngưỡng `PERM_BAN_THRESHOLD` (500 điểm), kết nối sẽ bị ngắt và địa chỉ IP bị lưu vào danh sách cấm.

Bảng 3.8 mô tả ma trận điểm số rủi ro do `IntrusionDetector` quy định đối với từng hành vi vi phạm.

Bảng 3.8: Ma trận điểm số rủi ro (Threat Score) của hệ thống phòng thủ

Loại vi phạm (ViolationType)	Điểm rủi ro	Mô tả hành vi tấn công
PORT_SCAN	100 điểm	Cố tình dò quét cổng hoặc gửi dữ liệu khi chưa hoàn tất bắt tay giao thức (Handshake).
INJECTION_ATTEMPT	30 điểm	Chèn mã độc JavaScript/HTML vào trường văn bản đầu vào.
INVALID_PACKET	10 điểm	Gửi tin sai cấu trúc Header hoặc vượt kích thước cho phép.
FAILED_AUTH	10 điểm	Đăng nhập thất bại do sai thông tin xác thực hoặc chữ ký điện tử không hợp lệ.

[PLACEHOLDER_HINH_ANH_KIEN_TRUC_XSS]

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

Đồ án đã hoàn thành mục tiêu xây dựng hệ thống phần mềm trò chuyện trực tuyến theo kiến trúc Client-Server. Hệ thống được phát triển bằng ngôn ngữ C++17, vận dụng các cơ sở lý thuyết về Mật mã học và An toàn thông tin vào thực tiễn.

Các kết quả chính bao gồm:

1. Về Kiến trúc mạng và Hiệu năng:

- Xây dựng máy chủ trên nền tảng Linux, ứng dụng cơ chế I/O phi đồng bộ qua API `epoll` kết hợp mô hình Thread-Pool. Kết quả đo lường (Stress Test) ghi nhận máy chủ duy trì thông lượng trên 9,000 gói tin/giây với độ trễ phản hồi ở phân vị 99% dưới 16ms.
- Phát triển ứng dụng máy khách hoạt động đa nền tảng (Linux, Windows). Giao thức truyền tải được thiết kế kết hợp Header nhị phân và Payload JSON nhằm chuẩn hóa quy trình đóng gói và phân giải dữ liệu.

2. Về An toàn thông tin:

- Tính Bảo mật (Confidentiality): Tích hợp cơ chế trao đổi khóa lai giữa phương pháp bất đối xứng RSA-2048 và đối xứng AES-256-GCM, mã hóa nội dung thông điệp tại tầng ứng dụng (E2EE).
- Tính Toàn vẹn (Integrity): Sử dụng hàm băm SHA-256 để xác thực mảnh tệp tin và ứng dụng cơ chế chuỗi khối (Blockchain-lite) vào hệ thống AuditLogger nhằm đảm bảo tính toàn vẹn của nhật ký kiểm toán.
- Tính Sẵn sàng (Availability): Triển khai hệ thống phòng thủ bao gồm IntrusionDetector và RateLimiter (Token Bucket) nhằm phân tích vi phạm và cách ly các nguồn phát tán lưu lượng bất thường.

3. Về Tính năng nghiệp vụ:

- Định tuyến thông điệp thời gian thực qua các phòng (Rooms) độc lập, hỗ trợ cơ chế bảo vệ bằng mật khẩu (băm qua PBKDF2).
- Xây dựng luồng truyền tải tệp tin qua trạm trung chuyển (Server-Relayed Transfer), kết hợp bộ lọc làm sạch dữ liệu (Sanitize) và tự động thu hồi tài nguyên (Garbage Collection).
- Tích hợp hệ thống kiểm soát truy cập dựa trên vai trò (RBAC) để thực thi các chính sách quản trị như cấm phát ngôn (/mute) và khóa tài khoản (/ban).

4.2 Hướng phát triển tương lai

Để hệ thống đáp ứng các yêu cầu mở rộng và tối ưu hóa tính năng trong thực tiễn, một số định hướng phát triển tiếp theo được đề xuất như sau:

1. Nâng cấp Giao thức Bảo mật chuyển tiếp hoàn hảo (Perfect Forward Secrecy - PFS): Hệ thống hiện tại sử dụng thuật toán RSA để mã hóa khóa phiên AES, chưa hỗ trợ PFS. Việc nâng cấp lên thuật toán trao đổi khóa Elliptic Curve Diffie-Hellman (ECDHE) sẽ đảm bảo mỗi phiên giao tiếp sử dụng một khóa độc lập. Cơ chế này ngăn chặn nguy cơ dữ liệu quá khứ bị giải mã ngay cả khi khóa bí mật dài hạn của máy chủ bị lộ.
2. Tích hợp Mã hóa đầu cuối (End-to-End Encryption - E2EE): Bên cạnh mã hóa tầng giao vận Client-Server, việc áp dụng Giao thức Signal (Signal Protocol) sẽ hỗ trợ mã hóa E2EE cho các phiên trò chuyện. Máy chủ sẽ hoạt động với vai trò trung chuyển (Blind Relay) đối với các luồng dữ liệu mã hóa, tăng cường mức độ bảo mật cho thông điệp người dùng.
3. Triển khai Kiến trúc Phân tán (Distributed Architecture): Để xử lý lưu lượng mạng vượt ngưỡng giới hạn của một máy chủ vật lý, kiến trúc nguyên khối (Monolithic) có thể được chuyển đổi sang mô hình cụm phân tán (Cluster/Microservices). Sự kết hợp với Redis Pub/Sub hoặc Apache Kafka làm hệ thống trung chuyển thông điệp (Message Broker) giữa các Node sẽ hỗ trợ phân tán tải và nâng cao khả năng mở rộng (Scalability) của hệ thống.
4. Phát triển Giao diện người dùng (GUI) và Giao thức Đa phương tiện: Chuyển đổi Client từ giao diện dòng lệnh (CLI) sang giao diện đồ họa thông qua bộ công cụ Qt (C++) hoặc Electron/Tauri. Dự án cũng có thể tích hợp tiêu chuẩn WebRTC nhằm cung cấp tính năng gọi thoại và gọi video theo mô hình mạng ngang hàng (P2P), giảm tải băng thông cho máy chủ trung tâm.